

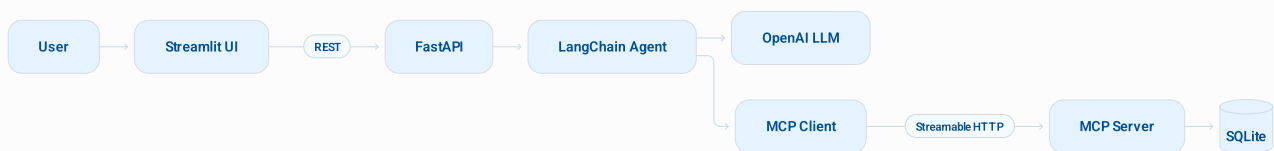
Sales MCP Application — Step-by-Step Development

1. Project objective

Build a Sales Assistant where users can:

- List products
- Check product stock and price
- Retrieve an order
- Calculate sales revenue
- Create a sales order after confirmation

Architecture:



Important communication paths:

- Streamlit calls FastAPI using REST.
- The agent calls OpenAI for reasoning.
- The agent accesses sales operations only through the MCP client.
- MCP server accesses the database.

Download the completed project:

[Download sales-mcp-app.zip](#)

2. Software requirements

Install:

- Python 3.11–3.13
- VS Code
- OpenAI API key
- Postman—optional

Python libraries:

```
mcp
fastapi
uvicorn
streamlit
langchain
langchain-openai
python-dotenv
requests
```

3. Create the project

Create a folder:

PowerShell

```
mkdir sales-mcp-app
cd sales-mcp-app
```

Create this structure:

```
sales-mcp-app/
├── .env
├── .env.example
├── .gitignore
├── config.toml
├── requirements.txt
├── run_mcp.py
├──
├── sales_app/
│   ├── __init__.py
│   ├── settings.py
│   ├── database.py
│   ├── mcp_server.py
│   ├── agent.py
│   ├── api.py
│   └── ui.py
├──
└── tests/
    └── test_database.py
```

Part 1: Environment setup

4. Create a virtual environment

On Windows PowerShell:

PowerShell

```
py -3.13 -m venv salesenv  
salesenv\Scripts\Activate.ps1
```

If script execution is blocked:

PowerShell

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass  
salesenv\Scripts\Activate.ps1
```

Upgrade pip:

PowerShell

```
python -m pip install --upgrade pip
```

5. Create requirements.txt

```
mcp[cli]>=1.26,<2  
fastapi>=0.116,<1  
uvicorn[standard]>=0.35,<1  
streamlit>=1.49,<2  
langchain[mcp]>=1.4,<2  
langchain-openai>=0.3,<2  
python-dotenv>=1.1,<2  
requests>=2.32,<3  
httpx>=0.28,<1
```

Install everything:

PowerShell

```
pip install -r requirements.txt
```

Verify:

PowerShell

```
pip list
```

6. Configure the OpenAI key

Create `.env`:

env

```
OPENAI_API_KEY=your_actual_openai_api_key
```

The key is loaded here:

Python

Run

```
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
```

It is passed to the model here:

Python

Run

```
model = ChatOpenAI(  
    model=MODEL,  
    temperature=TEMPERATURE,  
    api_key=OPENAI_API_KEY,  
)
```

Never place the actual key directly in Python code.

7. Create `config.toml`

TOML

```
[application]  
name = "Sales MCP Assistant"  
  
[database]  
path = "data/sales.db"  
  
[mcp]  
host = "127.0.0.1"  
port = 8001  
url = "http://127.0.0.1:8001/mcp"  
  
[api]  
host = "127.0.0.1"  
port = 8000  
url = "http://127.0.0.1:8000"
```

```
[agent]
model = "gpt-5-mini"
temperature = 0
```

Ports used:

Component	Port
Streamlit	8501
FastAPI	8000
MCP server	8001

Part 2: Configuration

8. Create `sales_app/settings.py`

Python

Run

```
import os
import tomllib
from pathlib import Path

from dotenv import load_dotenv

PROJECT_ROOT = Path(__file__).resolve().parents[1]

load_dotenv(PROJECT_ROOT / ".env")

with open(PROJECT_ROOT / "config.toml", "rb") as file:
    CONFIG = tomllib.load(file)

DATABASE_PATH = PROJECT_ROOT / CONFIG["database"]["path"]

MCP_HOST = CONFIG["mcp"]["host"]
MCP_PORT = CONFIG["mcp"]["port"]
MCP_URL = CONFIG["mcp"]["url"]

API_URL = CONFIG["api"]["url"]

MODEL = CONFIG["agent"]["model"]
TEMPERATURE = CONFIG["agent"]["temperature"]

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
```

Purpose:

- Loads `.env`
 - Loads `config.toml`
 - Provides shared settings to every component
-

Part 3: Database

9. Create `sales_app/database.py`

Start with the connection:

Python

Run

```
import sqlite3
from contextlib import contextmanager

from sales_app.settings import DATABASE_PATH

@contextmanager
def connection():
    DATABASE_PATH.parent.mkdir(
        parents=True,
        exist_ok=True,
    )

    db = sqlite3.connect(DATABASE_PATH)
    db.row_factory = sqlite3.Row

    try:
        yield db
        db.commit()

    except Exception:
        db.rollback()
        raise

    finally:
        db.close()
```

10. Create database tables

Add:

Python

Run

```
SCHEMA = """
CREATE TABLE IF NOT EXISTS products (
    product_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    category TEXT NOT NULL,
    unit_price REAL NOT NULL,
    stock INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS orders (
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_name TEXT NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL,
    unit_price REAL NOT NULL,
    total_amount REAL NOT NULL,
    order_date TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'CONFIRMED',
    FOREIGN KEY (product_id)
        REFERENCES products(product_id)
);
"""
```

Add sample products:

Python

Run

```
PRODUCTS = [
    (101, "Laptop Pro 14", "Computers", 120000, 20),
    (102, "27-inch Monitor", "Displays", 32000, 35),
    (103, "Wireless Keyboard", "Accessories", 4500, 80),
    (104, "USB-C Dock", "Accessories", 12500, 45),
]
```

Initialize the database:

Python

Run

```
def initialize_database():
    with connection() as db:
        db.executescript(SCHEMA)

        db.executemany(
            """
            INSERT OR IGNORE INTO products
            (product_id, name, category, unit_price, stock)
            VALUES (?, ?, ?, ?, ?)
            """,
```

```
        PRODUCTS,  
    )
```

The database will be created automatically at:

```
data/sales.db
```

11. Add database operations

List products:

Python

Run

```
def list_products(category=None):  
    with connection() as db:  
  
        if category:  
            rows = db.execute(  
                """  
                SELECT *  
                FROM products  
                WHERE lower(category) = lower(?)  
                ORDER BY product_id  
                """,  
                (category,),  
            ).fetchall()  
  
        else:  
            rows = db.execute(  
                "SELECT * FROM products ORDER BY product_id"  
            ).fetchall()  
  
    return [dict(row) for row in rows]
```

Get an order:

Python

Run

```
def get_order(order_id):  
    with connection() as db:  
        row = db.execute(  
            """  
            SELECT  
                o.*,  
                p.name AS product_name,  
                p.category  
            FROM orders o  
            JOIN products p  
            ON p.product_id = o.product_id
```



```

        WHERE o.order_id = ?
        """
        (order_id,),
    ).fetchone()

    return dict(row) if row else None

```

Sales summary:

Python

Run

```

def sales_summary(start_date, end_date):
    with connection() as db:
        row = db.execute(
            """
            SELECT
                COUNT(*) AS order_count,
                COALESCE(SUM(quantity), 0) AS units_sold,
                COALESCE(SUM(total_amount), 0) AS revenue
            FROM orders
            WHERE order_date BETWEEN ? AND ?
                AND status = 'CONFIRMED'
            """
            (start_date, end_date),
        ).fetchone()

    return {
        "start_date": start_date,
        "end_date": end_date,
        "order_count": row["order_count"],
        "units_sold": row["units_sold"],
        "revenue": row["revenue"],
    }

```

Part 4: MCP server

12. Create `sales_app/mcp_server.py`

Python

Run

```

from mcp.server.fastmcp import FastMCP

from sales_app import database

mcp = FastMCP("Sales MCP Server")

```

FastMCP creates the MCP server.

13. Define the product tool

Python

Run

```
@mcp.tool()
def list_products(
    category: str | None = None,
) -> list[dict]:
    """List products, prices and available stock."""

    return database.list_products(category)
```

MCP automatically generates a schema from:

Python

Run

```
category: str | None
```

Conceptual schema:

JSON

```
{
  "category": {
    "type": ["string", "null"]
  }
}
```

14. Define the order lookup tool

Python

Run

```
@mcp.tool()
def get_order(order_id: int) -> dict:
    """Retrieve one sales order by order ID."""

    order = database.get_order(order_id)

    if order is None:
        return {
            "found": False,
            "message": "Order was not found"
```

```
}
```

```
return order
```

In Python, use `False`, not JSON `false`:

Python

Run

```
"found": False
```

15. Define the sales-summary tool

Python

Run

```
@mcp.tool()
def get_sales_summary(
    start_date: str,
    end_date: str,
) -> dict:
    """Calculate sales between two YYYY-MM-DD dates."""

    return database.sales_summary(
        start_date,
        end_date,
    )
```

The LLM can generate arguments such as:

JSON

```
{
  "start_date": "2026-09-01",
  "end_date": "2026-09-30"
}
```

16. Define an order-creation tool

Python

Run

```
@mcp.tool()
def create_sales_order(
    customer_name: str,
    product_id: int,
    quantity: int,
    user_confirmed: bool,
```

```

) -> dict:
    """Create an order only after explicit confirmation."""

    if not user_confirmed:
        return {
            "created": False,
            "confirmation_required": True,
            "message": "Ask the user to confirm the order."
        }

    order = database.create_order(
        customer_name,
        product_id,
        quantity,
    )

    return {
        "created": True,
        "order": order,
    }

```

This tool modifies data, so confirmation is required.

17. Define MCP resources

A resource provides read-only context.

Product catalog:

Python

Run

```

import json

@mcp.resource("sales://catalog")
def product_catalog() -> str:
    """Current product catalog."""

    products = database.list_products()

    return json.dumps(
        products,
        indent=2,
    )

```

Order policy:

Python

Run

```
@mcp.resource("sales://policy/order-creation")
def order_creation_policy() -> str:
    """Sales-order creation policy."""

    return """
    Create an order only after the user explicitly
    confirms customer name, product ID and quantity.
    """
```

Tools perform operations. Resources provide contextual information.

18. Define an MCP prompt

Python

Run

```
@mcp.prompt()
def analyze_sales(
    start_date: str,
    end_date: str,
) -> str:
    """Reusable sales-analysis prompt."""

    return f"""
    Analyze confirmed sales from
    {start_date} through {end_date}.

    Use get_sales_summary.

    Report:
    1. Revenue
    2. Units sold
    3. Order count
    4. Top-performing product
    """
```

A prompt is a reusable template, not a tool.

19. Start the MCP server

At the bottom of `mcp_server.py`:

Python

Run

```
if __name__ == "__main__":
    mcp.run(
        transport="streamable-http",
```

```
        host="127.0.0.1",
        port=8001,
        stateless_http=True,
        json_response=True,
    )
```

Create `run_mcp.py`:

Python

Run

```
from sales_app.mcp_server import mcp
from sales_app.settings import MCP_HOST, MCP_PORT

if __name__ == "__main__":
    mcp.run(
        transport="streamable-http",
        host=MCP_HOST,
        port=MCP_PORT,
        stateless_http=True,
        json_response=True,
    )
```

Run it:

PowerShell

```
python run_mcp.py
```

MCP endpoint:

```
http://127.0.0.1:8001/mcp
```

Part 5: LangChain agent

20. Create `sales_app/agent.py`

Import the libraries:

Python

Run

```
from langchain.agents import create_agent
from langchain.mcp import MCPAdapter
from langchain_openai import ChatOpenAI

from sales_app.settings import (
    MCP_URL,
    MODEL,
```

```
OPENAI_API_KEY,  
    TEMPERATURE,  
)
```

Define instructions:

Python

Run

```
AGENT_INSTRUCTIONS = """  
You are a sales assistant.  
  
Rules:  
- Use MCP tools for sales facts.  
- Never invent prices, stock or revenue.  
- Display currency in INR.  
- Ask for confirmation before creating an order.  
- Call create_sales_order only after explicit confirmation.  
"""
```

Create the agent function:

Python

Run

```
async def run_agent(question: str):  
  
    if not OPENAI_API_KEY:  
        raise ValueError("OPENAI_API_KEY is missing")  
  
    llm = ChatOpenAI(  
        model=MODEL,  
        temperature=TEMPERATURE,  
        api_key=OPENAI_API_KEY,  
    )  
  
    async with MCPAdapter(MCP_URL) as adapter:  
  
        tools = await adapter.list_tools()  
  
        agent = create_agent(  
            model=llm,  
            tools=tools,  
            system_prompt=AGENT_INSTRUCTIONS,  
        )  
  
        result = await agent.ainvoke(  
            {  
                "messages": [  
                    {  
                        "role": "user",  
                        "content": question,  
                    }  
                ]  
            })
```

```

        ]
    }
)

final_message = result["messages"][-1]

return {
    "answer": final_message.content,
    "available_tools": [
        tool.name for tool in tools
    ],
}

```

The actual LLM execution occurs here:

Python

Run

```
result = await agent.ainvoke(...)
```

Internally, the agent follows:

```

User question
  ↓
LLM call
  ↓
LLM selects MCP tool
  ↓
MCP tool executes
  ↓
Tool result returns
  ↓
Second LLM call
  ↓
Final answer

```

Part 6: FastAPI

21. Create `sales_app/api.py`

Python

Run

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

from sales_app.agent import run_agent

```



```
app = FastAPI(  
    title="Sales MCP Agent API",  
    version="1.0.0",  
)
```

Define request and response schemas:

Python

Run

```
class AgentRequest(BaseModel):  
    question: str = Field(  
        min_length=1,  
        max_length=2000,  
    )  
  
class AgentResponse(BaseModel):  
    answer: str  
    available_tools: list[str]
```

Add health endpoint:

Python

Run

```
@app.get("/health")  
def health():  
    return {  
        "status": "ok"  
    }
```

Add agent endpoint:

Python

Run

```
@app.post(  
    "/agent/query",  
    response_model=AgentResponse,  
)  
async def query_agent(request: AgentRequest):  
  
    try:  
        result = await run_agent(  
            request.question  
        )  
  
        return AgentResponse(**result)  
  
    except Exception as error:  
        raise HTTPException(  
            status_code=500,
```

```
detail=str(error),
```

Start FastAPI:

PowerShell

```
uvicorn sales_app.api:app --reload --port 8000
```

Open Swagger:

```
http://127.0.0.1:8000/docs
```

Test request:

JSON

```
{
  "question": "List all products"
}
```

Part 7: Streamlit

22. Create `sales_app/ui.py`

Python

Run

```
import requests
import streamlit as st

from sales_app.settings import API_URL


st.set_page_config(
    page_title="Sales MCP Assistant",
    page_icon="📈",
)

st.title("📈 Sales MCP Assistant")

st.caption(
    "Streamlit → FastAPI → Agent → MCP → SQLite"
)
```

Create the input:

Python

Run

```
question = st.text_input(  
    "Ask a sales question",  
    value="List all available products",  
)
```

Call FastAPI:

Python

Run

```
if st.button("Submit"):  
  
    with st.spinner("Processing..."):  
  
        response = requests.post(  
            f"{API_URL}/agent/query",  
            json={  
                "question": question  
            },  
            timeout=120,  
        )  
  
        if response.status_code == 200:  
            result = response.json()  
  
            st.success("Completed")  
            st.write(result["answer"])  
  
            with st.expander(  
                "MCP tools discovered"  
            ):  
                st.write(  
                    result["available_tools"]  
                )  
  
        else:  
            st.error(response.text)
```

Start Streamlit:

PowerShell

```
streamlit run sales_app/ui.py
```

Open:

```
http://localhost:8501
```

Part 8: Start the complete application

Use three terminals with the virtual environment activated.

Terminal 1 — MCP server

PowerShell

```
cd sales-mcp-app
salesenv\Scripts\Activate.ps1
python run_mcp.py
```

Terminal 2 — FastAPI

PowerShell

```
cd sales-mcp-app
salesenv\Scripts\Activate.ps1
uvicorn sales_app.api:app --reload --port 8000
```

Terminal 3 — Streamlit

PowerShell

```
cd sales-mcp-app
salesenv\Scripts\Activate.ps1
streamlit run sales_app/ui.py
```

Start them in this order:

1. MCP server
2. FastAPI
3. Streamlit

Part 9: Test scenarios

Test 1: Product lookup

List all available products

Expected tool:

```
list_products
```

Test 2: Category lookup

```
Show Accessories products
```

Expected arguments:

JSON

```
{  
  "category": "Accessories"  
}
```

Test 3: Sales summary

```
Calculate sales from 2026-09-01 to 2026-09-30
```

Expected tool:

```
get_sales_summary
```

Test 4: Order lookup

```
Show order 1
```

Expected tool:

```
get_order
```

Test 5: Create an order

First message:

```
Create an order for Northwind Ltd,  
product 104, quantity 2
```

Expected result:

```
The agent asks for confirmation.
```

Confirmation message:

Confirm Northwind Ltd,
product 104, quantity 2

Expected tool call:

JSON

```
{
  "customer_name": "Northwind Ltd",
  "product_id": 104,
  "quantity": 2,
  "user_confirmed": true
}
```

Final application flow

